

Introducción

El Lab 3 te introducirá a las vulnerabilidades de Buffer overflow, en el contexto de un servidor web llamado **zookws**. El servidor web **zookws** ejecuta una aplicación web Python simple, **zoobar**, con la cual los usuarios transfieren “zoobars” (créditos) entre sí. Encontrarás Buffer overflows en el código del servidor web **zookws**, escribirás exploits para los Buffer overflows para inyectar código en el servidor a través de la red, y descubrirás cómo evadir la protección de pila no ejecutable. Los laboratorios posteriores analizarán otros aspectos de seguridad de la infraestructura **zoobar** y **zookws**.

Varios laboratorios, incluyendo este, te piden que diseñes exploits. Estos exploits son lo suficientemente realistas como para que puedas usarlos para un ataque real, pero **no debes hacerlo**. El punto de diseñar exploits es enseñarte cómo defenderte contra ellos, no cómo usarlos—atacar sistemas informáticos es ilegal y puede meterte en serios problemas. No lo hagas.

Comenzando

Los archivos que necesitarás para este y laboratorios posteriores se distribuyen usando el sistema de control de versiones Git. También puedes usar Git para hacer seguimiento de cualquier cambio que hagas al código fuente inicial. Aquí hay una visión general de Git y el manual del usuario de Git, que puedes encontrar útil. El repositorio Git del curso está disponible en <https://github.com/nachoparada/IIC2531-labs.git>. Para obtener el código del laboratorio, inicia sesión en la VM usando la cuenta `|student|` y clona el código fuente para el lab 1 de la siguiente manera:

```
student@6858-v22:~$ git clone https://github.com/nachoparada/IIC2531-labs.git lab
Cloning into 'lab'...
student@6858-v22:~$ cd lab
student@6858-v22:~/lab$
```

Antes de que sigas con este laboratorio, asegurate de que puedas compilar **zookws**:

```
student@6858-v22:~/lab$ make
cc zookd.c -c -o zookd.o -m64 -g -std=c99 -Wall -D\__GNU\__SOURCE -static -fno-stack-protector
cc http.c -c -o http.o -m64 -g -std=c99 -Wall -D\__GNU\__SOURCE -static -fno-stack-protector
cc -m64 zookd.o http.o -lcrypto -o zookd
cc -m64 zookd.o http.o -lcrypto -o zookd-exstack -z execstack
cc -m64 zookd.o http.o -lcrypto -o zookd-nxstack
cc zookd.c -c -o zookd-withssp.o -m64 -g -std=c99 -Wall -D\__GNU\__SOURCE -static
cc http.c -c -o http-withssp.o -m64 -g -std=c99 -Wall -D\__GNU\__SOURCE -static
cc -m64 zookd-withssp.o http-withssp.o -lcrypto -o zookd-withssp
cc -m64 -c -o shellcode.o shellcode.S
objcopy -S -O binary -j .text shellcode.o shellcode.bin
cc run-shellcode.c -c -o run-shellcode.o -m64 -g -std=c99 -Wall -D\__GNU\__SOURCE -static -fno-stack-protector
cc -m64 run-shellcode.o -lcrypto -o run-shellcode
rm shellcode.o
student@6858-v22:~/lab$
```

El componente de **zookws** que recibe solicitudes HTTP es **zookd**. Está escrito en C y sirve archivos estáticos y ejecuta scripts dinámicos. Para este laboratorio no tienes que entender los scripts dinámicos; están escritos en Python y los exploits en este laboratorio se aplican solo al código C. El código relacionado con HTTP está en **http.c**. Aquí hay un tutorial sobre el protocolo HTTP.

Hay dos versiones de **zookd** que usarás:

- **zookd-exstack**
- **zookd-nxstack**

zookd-exstack tiene una pila ejecutable, lo que facilita inyectar código ejecutable dada una vulnerabilidad de buffer overflow de pila. **zookd-nxstack** tiene una pila no ejecutable, y requiere un ataque más sofisticado para explotar buffer overflows de pila.

Para ejecutar el servidor web de manera predecible—de modo que su pila y diseño de memoria sea el mismo cada vez—usarás el script **clean-env.sh**. Esta es la misma manera en la que ejecutaremos el servidor web al revisar las tareas, así que asegurate de que todos tus exploits funcionen en esta configuración!

Los binarios de referencia de **zookd** se proporcionan en **bin.tar.gz**, que usaremos para revisar las tareas. Asegurate de que tus exploits funcionen en esos binarios. El comando **make check** siempre usará tanto **clean-env.sh** como **bin.tar.gz** para verificar tu entrega.

Ahora, asegúrate de que puedes ejecutar el servidor web `zookws` y acceder a la aplicación web `zobar` desde un navegador ejecutándose en tu máquina, de la siguiente manera:

```
student@6858-v22:~/lab$ ./clean-env.sh ./zookd 8080
```

El comando `./clean-env.sh` inicia `zookd` en el puerto 8080. Para abrir la aplicación `zobar`, abre tu navegador y apúntalo a la URL `http://DIRECCIONIP:8080/`, donde `DIRECCIONIP` es la dirección IP de la VM. Si algo no parece estar funcionando, intenta averiguar qué salió mal, o contacta al personal del curso, antes de proceder más lejos.

Parte 1: Encontrando Buffer overflows

En la primera parte de esta tarea de laboratorio, encontrarás Buffer overflows en el servidor web proporcionado. Para hacer este laboratorio, necesitarás entender los conceptos básicos de Buffer overflows. Para ayudarte a comenzar con esto, deberías leer *Smashing the Stack in the 21st Century*, que revisa los detalles de cómo funcionan los buffer overflows, y cómo pueden ser explotados.

Ejercicio 1. Estudia el código C del servidor web (en `zookd.c` y `http.c`), y encuentra un ejemplo de código que permita a un atacante sobrescribir la dirección de retorno de una función. Pista: busca buffers asignados en la pila. Escribe una descripción de la vulnerabilidad en el archivo `answers.txt`. Para tu vulnerabilidad, describe el buffer que puede desbordarse, cómo estructurarías la entrada al servidor web (es decir, la solicitud HTTP) para desbordar el buffer y sobrescribir la dirección de retorno, y la pila de llamadas que activará el Buffer overflow (es decir, la cadena de llamadas de función comenzando desde `process_client`).

Vale la pena tomar tu tiempo en este ejercicio y familiarizarte con el código, porque tu próximo trabajo es explotar la vulnerabilidad que identificaste. De hecho, puedes querer ir y venir entre este ejercicio y ejercicios posteriores, mientras trabajas los detalles y los documentas. Es decir, si encuentras un Buffer overflow que crees que puede ser explotado, puedes usar ejercicios posteriores para averiguar si de hecho puede ser explotado. Será útil dibujar un diagrama de pila como las figuras en *Smashing the Stack in the 21st Century*.

Ahora, comenzarás a desarrollar exploits para aprovechar los buffer overflows que has encontrado arriba. Hemos proporcionado código Python de para un exploit en `/home/student/lab/exploit-template.py`, que emite una solicitud HTTP. La plantilla de exploit toma dos argumentos, el nombre del servidor y el número de puerto, así que podrías ejecutarlo de la siguiente manera para emitir una solicitud a `zookws` ejecutándose en localhost:

```
student@6858-v22:~/lab$ ./clean-env.sh ./zookd-exstack 8080 &
[1] 2676
student@6858-v22:~/lab$ ./exploit-template.py localhost 8080
HTTP request:
b'GET / HTTP/1.0
```

```
'
```

```
...
```

```
student@6858-v22:~/lab$
```

Eres libre de usar este código, o escribir tu propio código de exploit desde cero. Ten en cuenta, sin embargo, que si eliges escribir tu propio exploit, el exploit debe ejecutarse correctamente dentro de la máquina virtual proporcionada.

Puedes encontrar `gdb` útil para construir tus exploits (aunque no es requerido que lo hagas). Como `zookd` crea muchos procesos (uno para cada cliente), puede ser difícil depurar el correcto. La forma más fácil de hacer esto es ejecutar el servidor web de antemano con `clean-env.sh` y luego adjuntar `gdb` a un proceso ya en ejecución con la bandera `-p`. Puedes encontrar el PID de un proceso usando `pgrep`; por ejemplo, para adjuntar a `zookd-exstack`, inicia el servidor y, en otra shell, ejecuta

```
student@6858-v22:~/lab$ gdb -p $(pgrep zookd-)
...
(gdb) break your-breakpoint
Breakpoint 1 at 0x1234567: file zookd.c, line 999.
(gdb) continue
Continuing.
```

Ten en cuenta que un proceso siendo depurado por `gdb` no será terminado incluso si terminas el proceso padre `zookd` usando `^C`. Si tienes problemas para reiniciar el servidor web, verifica si hay procesos sobrantes de la ejecución anterior, o asegúrate de salir de `gdb` antes de reiniciar `zookd`. También puedes ahorrarte algo de escritura usando `b` en lugar de `break`, y `c` en lugar de `continue`.

Cuando un proceso siendo depurado por `gdb` se bifurca, por defecto `gdb` continúa depurando el proceso padre y no se adjunta al hijo. Como `zookd` bifurca un proceso hijo para atender cada solicitud, puedes encontrar útil que `gdb` se adjunte al hijo en la

bifurcación, usando el comando `set follow-fork-mode child`. Hemos agregado ese comando a `/home/student/lab/.gdbinit`, que tendrá efecto si inicias `gdb` en ese directorio.

Mientras desarrollas tu exploit, puedes descubrir que causa que el servidor se quede pegado en lugar de caerse, dependiendo de qué Buffer overflow estés intentando aprovechar y qué datos estés sobrescribiendo en el servidor en ejecución. Puedes profundizar en los detalles de por qué se queda pegado, para entender cómo estás afectando la ejecución del servidor, para hacer que tu exploit evite esto y en su lugar bote el servidor. O puedes elegir explotar un Buffer overflow diferente que evite ese comportamiento.

Para este y ejercicios posteriores, puedes necesitar encodear tu *payload* de ataque de diferentes maneras, dependiendo de qué vulnerabilidad estés explotando. En algunos casos, puedes necesitar asegurarte de que tu *payload* de ataque esté encodeada en URL; es decir, usa `+` en lugar de espacio y `%2b` en lugar de `+`. Aquí hay una referencia de codificación URL y una herramienta de conversión útil. También puedes usar funciones de citado en el módulo Python `urllib` para codificar bytes en URL (en particular, `urllib.parse.quote_from_bytes`, seguido de `.encode('ascii')` para obtener los bytes de la cadena). En otros casos, puedes necesitar incluir valores binarios en tu carga útil. El módulo Python `struct` puede ayudarte a hacer eso. Por ejemplo, `struct.pack("<Q", x)` producirá un *encoding* binaria de 8 bytes (64 bits) del entero `x`.

Ejercicio 2. Escribe un exploit que use un Buffer overflow para botar el servidor web (o uno de los procesos que crea). No necesitas inyectar código en este punto. Verifica que tu *exploit* bote el servidor revisando las últimas líneas de `dmesg | tail`, usando `gdb`, u observando que el servidor web se cae (es decir, imprimirá `Child process 9999 terminated incorrectly, receiving signal 11`)

Proporciona el código para el *exploit* en un archivo llamado `exploit-2.py`.

La vulnerabilidad que encontraste en el Ejercicio 1 puede ser demasiado difícil de explotar. Siéntete libre de encontrar y explotar una vulnerabilidad diferente.

Puedes revisar si tus *exploits* botan el servidor de la siguiente manera:

```
student@6858-v22:~/lab$ make check-crash
```

Parte 2: Inyección de código

En esta parte, usarás tus exploits de Buffer overflow para inyectar código en el servidor web. El objetivo del código inyectado será desvincular (eliminar) un archivo sensible en el servidor, a saber `/home/student/grades.txt`. Usa `zookd-exstack`, ya que tiene una pila ejecutable que facilita inyectar código. El servidor web `zookws` debe iniciarse de la siguiente manera.

```
student@6858-v22:~/lab$ ./clean-env.sh ./zookd-exstack 8080
```

Puedes construir el exploit en dos pasos. Primero, escribe el código shell que desvincula el archivo sensible, a saber `/home/student/grades.txt`. Segundo, incrusta el código shell compilado en una solicitud HTTP que active el Buffer overflow en el servidor web.

Al escribir código shell, a menudo es más fácil usar lenguaje ensamblador en lugar de lenguajes de alto nivel, como C. Esto es porque el exploit generalmente necesita control fino sobre el diseño de la pila, valores de registros y tamaño del código. El compilador C generará preludios de función adicionales y realizará varias optimizaciones, lo que hace que el código binario compilado sea impredecible.

Hemos proporcionado código shell para que uses en `/home/student/lab/shellcode.S`, junto con reglas del `Makefile` que producen `/home/student/lab/shellcode.bin`, una versión compilada del código shell, cuando ejecutes `make`. El código shell proporcionado está destinado a explotar binarios `setuid-root`, y por lo tanto ejecuta un shell. Necesitarás modificar este código shell para en su lugar desvincular `/home/student/grades.txt`.

Para ayudarte a desarrollar tu código shell para este ejercicio, hemos proporcionado un programa llamado `run-shellcode` que ejecutará tu código shell binario, como si hubieras saltado correctamente a su punto de inicio. Por ejemplo, ejecutarlo en el código shell proporcionado hará que el programa ejecute `execve("/bin/sh")`, dándote así otro prompt de shell:

```
student@6858-v22:~/lab$ ./run-shellcode shellcode.bin
```

Ejercicio 3 (calentamiento). Modifica `shellcode.S` para desvincular `/home/student/grades.txt`. Tu código ensamblador puede invocar la llamada al sistema `SYS_unlink`, o llamar a la función de biblioteca `unlink()`.

Para probar si el código shell hace su trabajo, ejecuta los siguientes comandos:

```
student@6858-v22:~/lab$ make
student@6858-v22:~/lab$ touch ~/grades.txt
student@6858-v22:~/lab$ ./run-shellcode shellcode.bin
# Asegúrate de que /home/student/grades.txt haya desaparecido
```

```
student@6858-v22:~/lab$ ls ~/grades.txt
ls: cannot access /home/student/grades.txt: No such file or directory
```

Puedes encontrar `strace` útil cuando intentas averiguar qué llamadas al sistema está haciendo tu shellcode. Muy similar a `gdb`, adjuntas `strace` a un programa en ejecución:

```
student@6858-v22:~/lab$ strace -f -p $(pgrep zookd-)
```

Luego imprimirá todas las llamadas al sistema que hace ese programa. Si tu código shell no está funcionando, intenta buscar la llamada al sistema que crees que tu código shell debería estar ejecutando (es decir, `unlink`), y ve si tiene los argumentos correctos.

A continuación, construimos una solicitud HTTP maliciosa que inyecta el código byte compilado al servidor web, y secuestra el flujo de control del servidor para ejecutar el código inyectado. Al desarrollar un *exploit*, tendrás que pensar en qué valores están en la pila, para que puedas modificarlos en consecuencia.

Cuando estés construyendo un exploit, a menudo necesitarás conocer las direcciones de ubicaciones específicas de la pila, o funciones específicas, en un programa en particular. Una forma de hacer esto es agregar declaraciones `printf()` a la función en cuestión. Por ejemplo, puedes usar `printf("Pointer: %p", &x);` para imprimir la dirección de la variable `x` o función `x`. Sin embargo, este enfoque requiere cierto cuidado: necesitas asegurarte de que las declaraciones que agregaste no estén cambiando por sí mismas el diseño de la pila o el diseño del código. Nosotros (y `make check`) calificaremos el laboratorio sin ninguna declaración `printf` que puedas haber agregado.

Un enfoque más a prueba de errores para determinar direcciones es usar `gdb`. Por ejemplo, supón que quieres conocer la dirección de pila del array `pn[]` en la función `http_serve` en `zookd-exstack`, y la dirección de su puntero de retorno guardado. Puedes obtenerlos usando `gdb` primero iniciando el servidor web (recuerda `clean-env!`), y luego adjuntando `gdb` a él:

```
student@6858-v22:~/lab$ gdb -p $(pgrep zookd-)
...
(gdb) break http\_serve
Breakpoint 1 at 0x5555555561c4: file http.c, line 275.
(gdb) continue
Continuing.
```

Asegúrate de ejecutar `gdb` desde el directorio `~/lab`, para que tome el comando `set follow-fork-mode child` de `~/lab/.gdbinit`. Ahora puedes emitir una solicitud HTTP al servidor web, para que active el breakpoint, y para que puedas examinar la pila de `http_serve`.

```
student@6858-v22:~/lab$ curl localhost:8080
```

Esto hará que `gdb` alcance el breakpoint que estableciste y detenga la ejecución, y te dará la oportunidad de preguntarle a `gdb` por las direcciones que te interesan:

```
Thread 2.1 "zookd-exstack" hit Breakpoint 1, http\_serve (fd=4, name=0x555555575fcec "/") at http.c:275
275      void (*handler)(int, const char *) = http\_serve\_none;
(gdb) print &pn
$1 = (char (*)[2048]) 0x7fffffff4a0
(gdb) info frame
Stack level 0, frame at 0x7fffffffcd0:
  rip = 0x5555555561c4 in http\_serve (http.c:275); saved rip = 0x55555555587b
  called by frame at 0x7fffffffed00
  source language c.
Arglist at 0x7fffffffcd0, args: fd=4, name=0x555555575fcec "/"
Locals at 0x7fffffffcd0, Previous frame's sp is 0x7fffffffcd0
Saved registers:
  rbx at 0x7fffffffcb8, rbp at 0x7fffffffcd0, rip at 0x7fffffffcd8
(gdb)
```

De esto, puedes decir que, al menos para esta invocación de `http_serve`, el buffer `pn[]` en la pila vive en la dirección `0x7fffffff4a0`, y el valor guardado de `%rip` (la dirección de retorno en otras palabras) está en `0x7fffffffcd8`. Si quieres ver el contenido de los registros, también puedes usar `info registers`.

Ahora es tu turno de desarrollar un exploit.

Ejercicio 4. Comenzando desde uno de tus exploits del Ejercicio 2, construye un exploit que secuestre el flujo de control del servidor web y desvincule `/home/student/grades.txt`. Guarda este exploit en un archivo llamado

`exploit-4.py`.

Verifica que tu exploit funcione; necesitarás recrear `/home/student/grades.txt` después de cada ejecución exitosa del exploit.

Sugerencia: primero enfócate en obtener control del contador del programa. Dibuja el diseño de la pila que esperas que el programa tenga en el punto cuando desbordes el buffer, y usa `gdb` para verificar que tus datos de desbordamiento terminen donde esperas que estén. Avanza paso a paso por la ejecución de la función hasta la instrucción de retorno para asegurarte de que puedes controlar a qué dirección el programa retorna. Los comandos `next`, `stepi`, y `x` en `gdb` deberían resultar útiles.

Una vez que puedas secuestrar de manera confiable el flujo de control del programa, encuentra una dirección adecuada que contendrá el código que quieres ejecutar, y enfócate en colocar el código correcto en esa dirección—por ejemplo, una derivación del código shell proporcionado.

Puedes asegurarte de que tu *exploit* funciona de la siguiente manera:

```
student@6858-v22:~/lab$ make check-exstack
```

La prueba imprime “PASS” o “FAIL”. Calificaremos tus exploits de esta manera. No cambies el `Makefile`.

El compilador C estándar usado en Linux, `gcc`, implementa una versión de canarios de pila (llamada SSP). Puedes explorar si la versión de GCC de canarios de pila prevendría o no una vulnerabilidad dada usando las versiones habilitadas para SSP de `zookd`: `zookd-withssp`.

Parte 3: Ataques de retorno a `libc`

Muchos sistemas operativos modernos marcan la pila como no ejecutable en un intento de hacer más difícil explotar Buffer overflows. En esta parte, explorarás cómo se puede evadir este mecanismo de protección. Ejecuta el servidor web configurado con binarios que tienen una pila no ejecutable, de la siguiente manera.

```
student@6858-v22:~/lab$ ./clean-env.sh ./zookd-nxstack 8080
```

La observación clave para explotar Buffer overflows con una pila no ejecutable es que aún controlas el contador del programa, después de que una instrucción `ret` salte a una dirección que colocaste en la pila. Aunque no puedes saltar a la dirección del buffer desbordado (no será ejecutable), usualmente hay suficiente código en el espacio de direcciones del servidor vulnerable para realizar la operación que quieres.

Así, para evadir una pila no ejecutable, necesitas primero encontrar el código que quieres ejecutar. Esto a menudo es una función en la biblioteca estándar, llamada `libc`, como `execve`, `system`, o `unlink`. Luego, necesitas arreglar para que la pila y los registros estén en un estado consistente con llamar a esa función con los argumentos deseados. Finalmente, necesitas arreglar para que la instrucción `ret` salte a la función que encontraste en el primer paso. Este ataque a menudo se llama un ataque de *retorno a libc*.

Un desafío con los ataques de *retorno a libc* es que necesitas pasar argumentos a la función `libc` que quieres invocar. Las convenciones de llamada x86-64 hacen que esto sea un desafío porque los primeros 6 argumentos se pasan en registros. Por ejemplo, el primer argumento debe estar en el registro `%rdi` (ver `man 2 syscall`, que documenta la convención de llamada). Así, necesitas una instrucción que cargue el primer argumento en `%rdi`. En el Ejercicio 3, podrías haber puesto esa instrucción en el buffer que tu exploit desborda. Pero, en esta parte del laboratorio, la pila está marcada como no ejecutable, así que ejecutar la instrucción haría que el servidor se bloquee, pero no ejecutaría la instrucción.

La solución a este problema es encontrar una pieza de código en el servidor que cargue una dirección en `%rdi`. Tal pieza de código se refiere como un “fragmento de código prestado”, o más generalmente como un *gadget rop*, porque es una herramienta para programación orientada a retorno (rop). Afortunadamente, `zookd.c` accidentalmente tiene un gadget útil: ve la función `accidentally`.

Ejercicio 5. Comenzando desde tu exploit en los Ejercicios 2 y 4, construye un exploit que desvincule `/home/student/grades.txt` cuando se ejecute en los binarios que tienen una pila no ejecutable. Nombra este nuevo exploit `exploit-5.py`.

En este ataque vas a tomar control del servidor a través de la red *sin inyectar ningún código* en el servidor. Deberías usar un ataque de retorno a `libc` donde redirijas el flujo de control al código que ya existía antes de tu ataque. El esquema del ataque es realizar un buffer overflow que:

1. cause que el argumento a la función `libc` elegida esté en la pila
2. luego cause que `accidentally` se ejecute para que ese argumento termine en `%rdi`
3. y luego cause que `accidentally` retorne a la función `libc` elegida

Será útil dibujar un diagrama de pila como las figuras en Smashing the Stack in the 21st Century en (1) el punto que el Buffer overflow ocurre y (2) en el punto que **accidentally** se ejecuta.

Puedes probar tus exploits de la siguiente manera:

```
student@6858-v22:~/lab$ make check-libc
```

Parte 4: Arreglando Buffer overflows y otros errores

Ahora que has averiguado cómo explotar Buffer overflows, intentarás encontrar otros tipos de vulnerabilidades en el mismo código. Como con muchas aplicaciones del mundo real, la “seguridad” de nuestro servidor web no está bien definida. Así, necesitarás usar tu imaginación para pensar en un modelo de amenaza plausible y política para el servidor web.

Ejercicio 6. Mira a través del código fuente e intenta encontrar más vulnerabilidades que puedan permitir a un atacante comprometer la seguridad del servidor web. Describe los ataques que has encontrado en `answers.txt`, junto con una explicación de las limitaciones del ataque, lo que un atacante puede lograr, por qué funciona, y cómo podrías ir sobre arreglarlo o prevenirlo. Deberías ignorar errores en el código de `zoobar`. Serán abordados en laboratorios futuros.

Un enfoque para encontrar vulnerabilidades es rastrear el flujo de entradas controladas por el atacante a través del código del servidor. En cada punto que la entrada del atacante se use, considera todos los valores posibles que el atacante podría haber proporcionado en ese punto, y lo que el atacante puede lograr de esa manera.

Deberías encontrar al menos dos vulnerabilidades para este ejercicio.

Finalmente, arreglarás las vulnerabilidades que has explotado en esta tarea de laboratorio.

Ejercicio 7. Para cada vulnerabilidad de Buffer overflow que hayas explotado en los Ejercicios 2, 4, y 5, arregla el código del servidor web para prevenir la vulnerabilidad en primer lugar. No dependas de mecanismos de compilación o tiempo de ejecución como canarios de pila, removiendo `-fno-stack-protector`, verificación de límites baggy, etc.

Asegúrate de que tu código realmente detenga tus exploits de funcionar. Usa `make check-fixed` para ejecutar tus exploits contra tu código fuente modificado (en oposición a los binarios de referencia del personal de `bin.tar.gz`). Estas verificaciones deberían reportar FAIL (es decir, el exploit ya no funciona). Si reportan PASS, esto significa que el exploit aún funciona, y no arreglaste correctamente la vulnerabilidad.

Nota que tu entrega *no* debería hacer cambios al `Makefile` y otros scripts de calificación. Usaremos nuestra versión no modificada durante la calificación.

También deberías asegurarte de que tu código aún pase todas las pruebas usando `make check`, que usa los binarios de laboratorio no modificados.

¡Has terminado! Entrega tus respuestas a la tarea de laboratorio ejecutando `prepare-submit` y subiendo el archivo resultante `lab1-handin.tar.gz` al buzón de la tarea.